Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23EC5PE1AD]

Submitted by

Shashwath S Parol **1BM24EC191**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

Contents

No.	Program
1	Arithmetic Logic Unit (ALU)
2	Comparator
3	4-Bit Parity Generator and Checker
4	4-Bit Barrel Shifter
5	Timer Module
6	Ring Counter
7	Johnson Counter
8	Synchronous and Asynchronous Counters
9	Shift Register
10	Divide-by-N Counter
11	Mealy FSM Example
12	Moore FSM Example
13	Random Sequence Generator

1. Arithmetic Logic Unit (ALU)

Theory

An Arithmetic Logic Unit (ALU) is the core mathematical and decision-making engine of a central processing unit (CPU). It executes operations governing data manipulation, control flow, and system decision-making. Structurally, an ALU relies on combinational logic split into arithmetic, logic, and relational comparison sections operating in parallel.

Operation routing is managed by a selection control bus (sel). This select line acts as the address input for multiplexers or case statements, steering the requested sub-operation result to the primary output data bus. If an invalid or unmapped select code is given, fallback logic defaults the output to zero to avoid floating signals.

Beyond raw data outputs, an ALU generates status flags reflecting result properties. The most prominent is the zero flag, which asserts high if all output bits equal zero. Other flags—such as carry, sign, and overflow—provide immediate feedback for conditional branching and CPU status register updates.

Design

4-bit ALU

Arithmetic Logic Unit

Performs basic arithmetic and logical operations on 4-bit inputs.

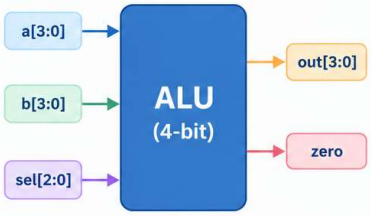
Module Overview

This 4-bit ALU takes two 4-bit inputs (a and b) and a 3-bit select signal (sel) to perform one of eight operations.

The result is a 4-bit output along with a zero flag that indicates whether the output is zero.

a	: 4-bit input
b	: 4-bit input
sel	: 3-bit select
out	: 4-bit output
zero	: Zero flag (1 if out = 0)

Block Diagram



Supported Operations

sel[2:0]	Operation	Description
000	a + b	Addition
001	a – b	Subtraction
010	a * b	Multiplication
011	{4'b0000, (a & b)}	AND (zero-extended)
100	{4'b0000, (a | b)}	OR (zero-extended)
101	{4'b0000, (a ^ b)}	XOR (zero-extended)
110	a << b	Left Shift (by b)
111	~a	Bitwise NOT (zero-extended)

How It Works

1. Takes two 4-bit inputs (a and b) and a 3-bit select signal (sel).
2. Performs the selected operation and produces an 8-bit result (internally).
3. The upper 4 bits are used for logical operations and shifts.
4. The zero flag is high when the 8-bit output is 0.

```systemverilog
module tb_alu;

  logic [3:0] a;
  logic [3:0] b;
  logic [2:0] sel;
  logic [7:0] out;
  logic       zero;

  alu uut (
    .a(a),
    .b(b),
    .sel(sel),
    .out(out),
    .zero(zero)
  );

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_alu);

    a = 4'd6; b = 4'd3;

    sel = 3'b000; #10;
    sel = 3'b001; #10;
    sel = 3'b010; #10;
    sel = 3'b011; #10;
    sel = 3'b100; #10;
    sel = 3'b101; #10;
    sel = 3'b110; #10;
    sel = 3'b111; #10;

    a = 4'd0; b = 4'd0;
    sel = 3'b000; #10;

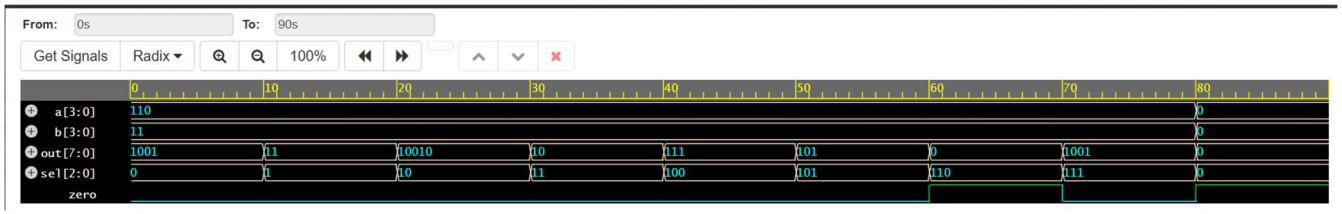
    $finish;
  end
```

```systemverilog
module alu (
    input  logic [3:0] a,
    input  logic [3:0] b,
    input  logic [2:0] sel,
    output logic [7:0] out,
    output logic       zero
);

  always_comb begin
    out = 8'h00;
    case (sel)
      3'b000: out = a + b;
      3'b001: out = a - b;
      3'b010: out = a * b;
      3'b011: out = {4'b0000, (a & b)};
      3'b100: out = {4'b0000, (a | b)};
      3'b101: out = {4'b0000, (a ^ b)};
      3'b110: out = (a < b) ? 8'h01 : 8'h00;
      3'b111: out = {4'b0000, ~a};
      default: out = 8'h00;
    endcase

    zero = (out == 8'h00) ? 1'b1 : 1'b0;
  end

endmodule
```



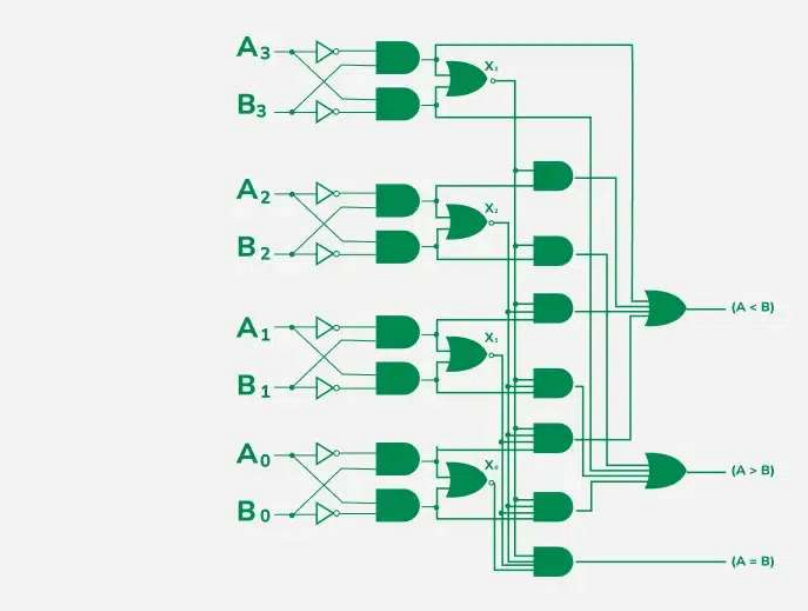
2. Comparator

Theory

A digital comparator is a combinational logic circuit that evaluates the relative magnitude of two binary numbers to ascertain their relationship. It determines whether one input operand is greater than, less than, or equal to the other, yielding distinct binary output flags for each condition.

At the underlying gate level, comparators utilize logic primitives and subtraction networks. Equality detection is achieved by passing corresponding bits through XNOR gates, followed by an AND reduction network to verify that every bit position matches identically. Magnitude determination—identifying greater-than or less-than conditions—evaluates the input bits progressively starting from the most significant bit (MSB) down to the least significant bit (LSB). If a bit mismatch occurs at a higher-order position, the comparison resolves immediately regardless of lower-order values. They are heavily utilized in arithmetic execution units for conditional branching, memory address boundary checking, loop index tracking, and industrial process controllers where real-time sensor measurements are evaluated against reference threshold values.

Design



Verilog HDL Code

```verilog
module tb_comparator;

  logic [3:0] a;
  logic [3:0] b;
  logic       eq;
  logic       lt;
  logic       gt;

  comparator uut (
    .a(a),
    .b(b),
    .eq(eq),
    .lt(lt),
    .gt(gt)
  );

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_comparator);

    a = 4'd5; b = 4'd5; #10;
    a = 4'd3; b = 4'd7; #10;
    a = 4'd9; b = 4'd4; #10;
    a = 4'd0; b = 4'd0; #10;

    $finish;
  end

  initial begin
    $monitor("Time=%0t | a=%0d, b=%0d | eq=%b, lt=%b, gt=%b",
             $time, a, b, eq, lt, gt);
  end
endmodule
```

```verilog
module comparator (
    input  logic [3:0] a,
    input  logic [3:0] b,
    output logic       eq,
    output logic       lt,
    output logic       gt
);

  always_comb begin
    eq = (a == b);
    lt = (a < b);
    gt = (a > b);
  end

endmodule
```

3. 4-Bit Parity Generator and Checker

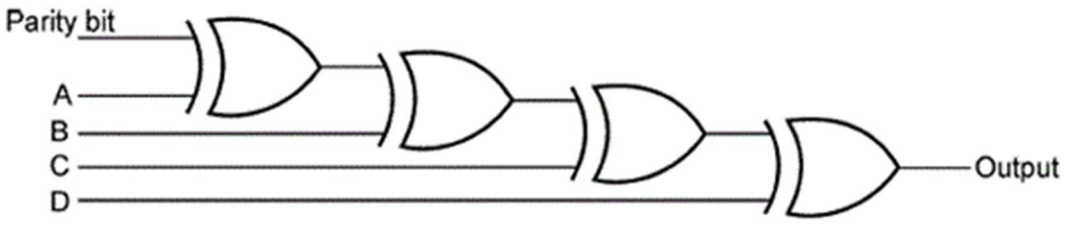
Theory

A parity generator and checker circuit is a fundamental error-detection mechanism used in digital communication and data storage systems to identify single-bit corruption during transmission or retrieval.

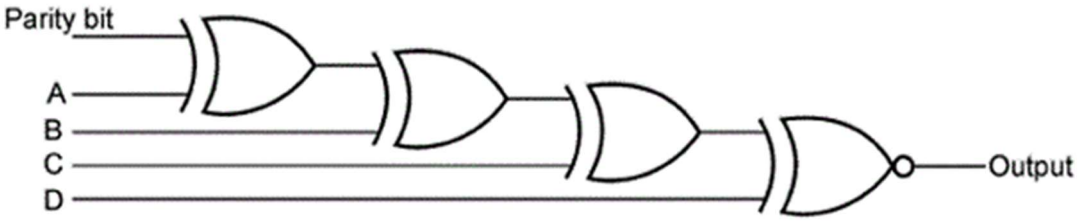
The core principle relies on appending an extra redundancy bit—the parity bit—to a data word. Parity can be implemented as either even parity (where the total number of binary ones in the combined data and parity word is even) or odd parity (where the total count of ones is odd). At the hardware level, the generation and checking processes are efficiently constructed using a tree of XOR (Exclusive-OR) gates. Because an XOR gate outputs high when an odd number of its inputs are high, cascading XOR gates computes the parity bit of a multi-bit word with minimal propagation delay.

- Parity Generator: Located at the transmitting end, this circuit evaluates the input data bits using XOR logic to produce the required parity bit, which is then sent alongside the data.
- Parity Checker: Located at the receiving end, this circuit recalculates the parity of the incoming data and compares it against the received parity bit using an XOR operation. If a mismatch occurs, the circuit asserts an error flag (chk_error), alerting the system that data corruption took place. While simple parity schemes cannot correct errors or detect multi-bit corruption, they provide lightweight, fast protection for basic digital hardware interfaces.

Design



ODD PARITY



EVEN PARITY

Verilog HDL Code

```
module tb_parity_gen_chk;
  wire  [3:0] data_in;
  wire        parity_type;
  reg       gen_parity;
  reg       chk_error;

  parity_gen_chk uut (
    .data_in(data_in),
    .parity_type(parity_type),
    .gen_parity(gen_parity),
    .chk_error(chk_error)
  );

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_parity_gen_chk);

    data_in = 4'b1011; parity_type = 1'b0; #10;
    data_in = 4'b1011; parity_type = 1'b1; #10;
    data_in = 4'b1100; parity_type = 1'b0; #10;
    data_in = 4'b1100; parity_type = 1'b1; #10;
    $finish;
  end

  initial begin
    $monitor("Time=%0t | data=%b, type=%b | gen=%b, error=%b",
             $time, data_in, parity_type, gen_parity, chk_error);
  end
endmodule
```

```
module parity_gen_chk (
    input [3:0] data_in,
    input parity_type, // 0 for even, 1 for odd
    output reg       gen_parity,
    output reg       chk_error
);

  reg calculated_parity;

  always @(*) begin
    calculated_parity = ^data_in;
    gen_parity = parity_type ? ~calculated_parity : calculated_parity;
    chk_error = gen_parity ^ calculated_parity;
  end

endmodule
```

4. 4-Bit Barrel Shifter

Theory

A barrel shifter is a combinational digital logic circuit that can shift or rotate a data word by a specified number of bit positions in a single operation, without the multi-cycle delay associated with sequential shift registers.

Unlike traditional shift registers that move data across flip-flops one clock edge at a time, a barrel shifter uses an array of multiplexers arranged in stages or crossbar structures. For a 4-bit or N-bit word, control lines dictate how many places the data shifts (e.g., by 0, 1, 2, or 3 positions). The multiplexer network routes each input bit directly to its calculated destination wire simultaneously during a single propagation delay.

Barrel shifters support several types of operations depending on architectural requirements:

- Logical Shift: Vacated bit positions are filled with zeros, used for unsigned multi-bit scaling.
- Arithmetic Shift: Preserves the most significant sign bit during right shifts to maintain proper negative values.
- Circular Rotate: Bits shifted off one end wrap around to fill the vacant positions on the opposite end.

They are critical components inside arithmetic logic units (ALUs), floating-point processors, and microprocessor data paths, enabling high-speed multiplication, division, and bit-manipulation instructions in a single clock cycle.

Design

4-bit Barrel Shifter

Rotates a 4-bit input left or right based on a 2-bit control signal.

A barrel shifter can perform multiple shift/rotate operations in parallel using multiplexers (combinational logic).

Overview

This 4-bit barrel shifter takes a 4-bit input (a) and a 2-bit select signal (sel) and produces a 4-bit output (out) by performing one of four operations:

- `00` : No shift (a)
- `01` : Shift left by 1 (rotate left)
- `10` : Shift left by 2 (rotate left)
- `11` : Shift right by 1 (rotate right)

Block Diagram



How It Works

1. The 2-bit select signal (sel) chooses the operation.
2. The input (a) is routed through multiplexers to perform the required shift/rotate.
3. The result is a 4-bit output (out).
4. All operations are done in parallel (using 4 layers of 2:1 multiplexers).

Operation Table

sel[1:0]	Operation	Description	Example (a = 4'b1010)
00	No shift	Output is the same as input.	out = 1010
01	Shift left by 1 (rotate left)	MSB moves to LSB.	out = 0101
10	Shift left by 2 (rotate left)	Bits rotate left by 2 positions.	out = 1010
11	Shift right by 1 (rotate right)	LSB moves to MSB.	out = 0101

Key Points

- Supports 4 different shift/rotate operations.
- Uses multiplexers for parallel operation.
- Simple, fast and efficient combinational logic design.

Verilog HDL Code

```
module tb_barrel_shifter;

  reg [3:0] data_in;
  reg [1:0] shift_amt;
  reg       dir;
  reg [3:0] data_out;

  barrel_shifter uut (
    .data_in(data_in),
    .shift_amt(shift_amt),
    .dir(dir),
    .data_out(data_out)
  );
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_barrel_shifter);

    data_in = 4'b1011; dir = 1'b0;
    shift_amt = 2'b00; #10;
    shift_amt = 2'b01; #10;
    shift_amt = 2'b10; #10;
    shift_amt = 2'b11; #10;

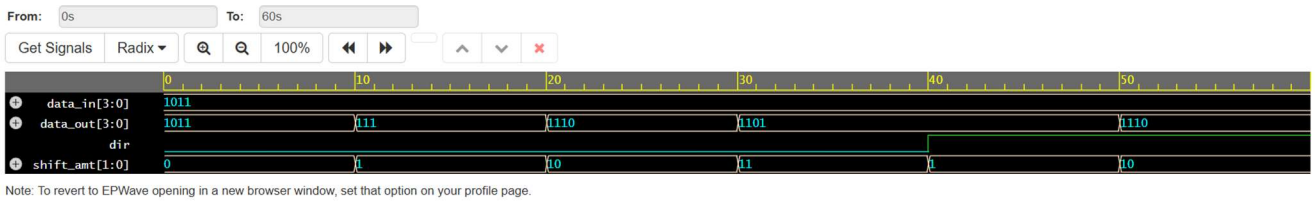
    dir = 1'b1;
    shift_amt = 2'b01; #10;
    shift_amt = 2'b10; #10;

    $finish;
  end
  initial begin
    $monitor("Time=%0t | data_in=%b, dir=%b, shift=%0d | data_out=%b",
             $time, data_in, dir, shift_amt, data_out);
  end
endmodule
```

```
module barrel_shifter (
    input [3:0] data_in,
    input [1:0] shift_amt,
    input dir,
    output reg [3:0] data_out
);

  always @(*) begin
    if (dir == 1'b0) begin
      case (shift_amt)
        2'b00: data_out = data_in;
        2'b01: data_out = {data_in[2:0], data_in[3]};
        2'b10: data_out = {data_in[1:0], data_in[3:2]};
        2'b11: data_out = {data_in[0], data_in[3:1]};
      endcase
    end else begin
      case (shift_amt)
        2'b00: data_out = data_in;
        2'b01: data_out = {data_in[0], data_in[3:1]};
        2'b10: data_out = {data_in[1:0], data_in[3:2]};
        2'b11: data_out = {data_in[2:0], data_in[3]};
      endcase
    end
  end

endmodule
```



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

5. Timer Module

Theory

A timer module is a sequential digital hardware circuit designed to track elapsed clock cycles and assert a timeout or interrupt signal once a pre-configured threshold count is reached. Timers are essential components in embedded systems, microcontrollers, and communication protocols for generating precise time delays, managing timeouts, and triggering periodic tasks.

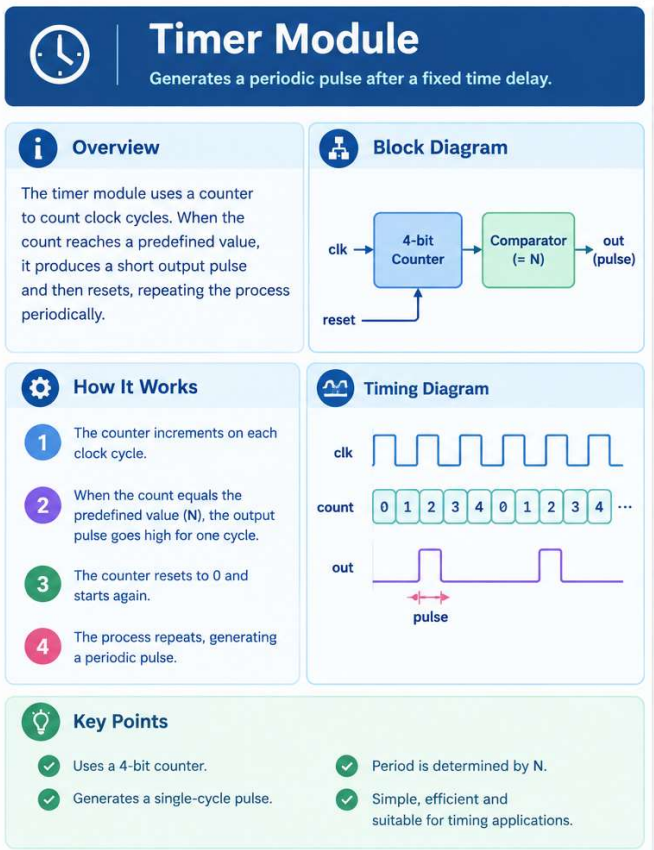
Operational Mechanics

The core architecture of a digital timer consists of a synchronous counter paired with a comparator or conditional threshold check:

- Enable Control: When the enable signal is asserted high, the internal counter increments its value on each active clock edge. When disabled, the counter typically holds its current value or resets depending on the design specification.
- Limit Comparison: As the counter increments, its current value is continuously compared against a preset limit register. Once the counter matches or exceeds this limit, the timer asserts a timeout flag high and resets the counter back to zero on the subsequent clock cycle.
- Asynchronous Reset: A reset signal forces both the counter register and the output timeout flag back to their initial default states (0), ensuring predictable startup behavior across the hardware.

Timers can be configured as down-counters or up-counters depending on whether they start from a maximum value and decrement to zero or start at zero and increment up to a maximum limit. They form the underlying backbone for watchdog timers, pulse-width modulation (PWM) generators, baud rate generators, and real-time operating system (RTOS) tick schedulers**.**

Design



Verilog HDL Code

```
module tb_timer_module;
  reg       clk;
  reg       rst;
  reg       enable;
  reg [3:0] limit;
  reg [3:0] count;
  reg       timeout;

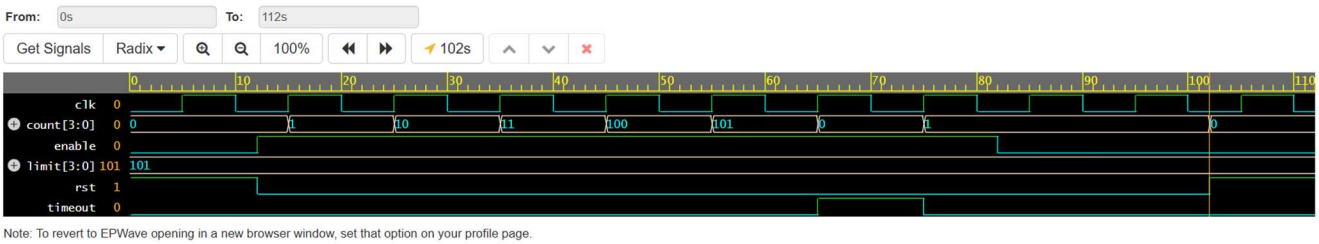
  timer_module uut (
    .clk(clk),
    .rst(rst),
    .enable(enable),
    .limit(limit),
    .count(count),
    .timeout(timeout)
  );

  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_timer_module);
    clk = 1'b0;
    rst = 1'b1;
    enable = 1'b0;
    limit = 4'd5;
    #12 rst = 1'b0; enable = 1'b1;
    #70 enable = 1'b0;
    #20 rst = 1'b1;
    #10 $finish;
  end
  initial begin
    $monitor("Time=%0t | rst=%b, enable=%b, limit=%0d | count=%0d, timeout=%b", $time, rst, enable, limit, count, timeout);
  end
endmodule
```

```
    input rst,
    input enable,
    input [3:0] limit,
    output reg [3:0] count,
    output reg       timeout
);

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      count <= 4'd0;
      timeout <= 1'b0;
    end
    else if (enable) begin
      if (count >= limit)
        begin
            count <= 4'd0;
            timeout <= 1'b1;
        end
      else
        begin
          count <= count + 1'b1;
          timeout <= 1'b0;
        end
      end
    else
      begin
        timeout <= 1'b0;
      end
  end

endmodule
```

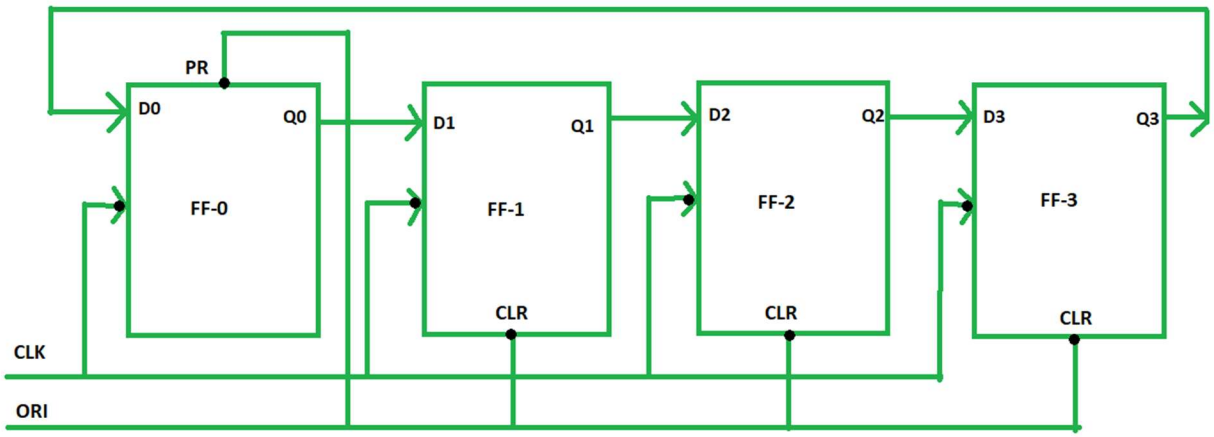


5. Ring Counter

Theory

A ring counter is a type of shift register counter where the output of the final flip-flop is fed back directly into the serial input of the first flip-flop. Unlike standard binary counters that cycle through a full binary progression, a ring counter circulates a single active bit (a hot-one code) around a closed loop of flip-flops.State Configuration: For an N-bit ring counter, the register cycles through N distinct states. Upon an asynchronous reset (rst), the counter is typically initialized to a known starting state, such as 4'b0001, where only the first stage holds a logic high (1) and all other stages hold a logic low (0).Circular Shift: On each subsequent rising edge of the clock (clk), the bits shift sequentially down the register chain. The high bit moves from position to position, and upon reaching the final stage, it wraps back around to the first input.State Sequence: For a 4-bit ring counter, the output pattern follows the exact sequence: 0001 → 0010 → 0100 → 1000 → and repeats back to 0001.

Design:



Verilog HDL Code

```verilog
module tb_ring_counter;

  reg       clk;
  reg       rst;
  reg [3:0] q;

  ring_counter uut (
    .clk(clk),
    .rst(rst),
    .q(q)
  );

  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_ring_counter);

    clk = 1'b0;
    rst = 1'b1;
    #12;

    rst = 1'b0;
    #60;

    $finish;
  end

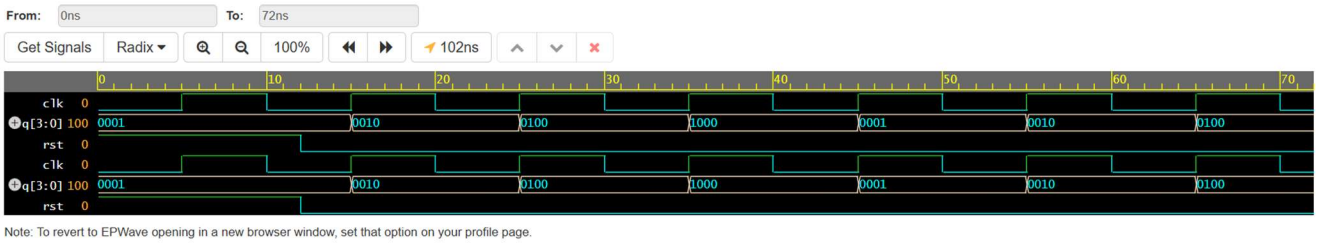
  initial begin
    $monitor("Time=%0t | rst=%b | q=%b", $time,
rst, q);
  end

endmodule
```

```verilog
module ring_counter (
    input clk,
    input rst,
    output reg [3:0] q
);

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      q <= 4'b0001;
    end else begin
      q <= {q[2:0], q[3]};
    end
  end

endmodule
```



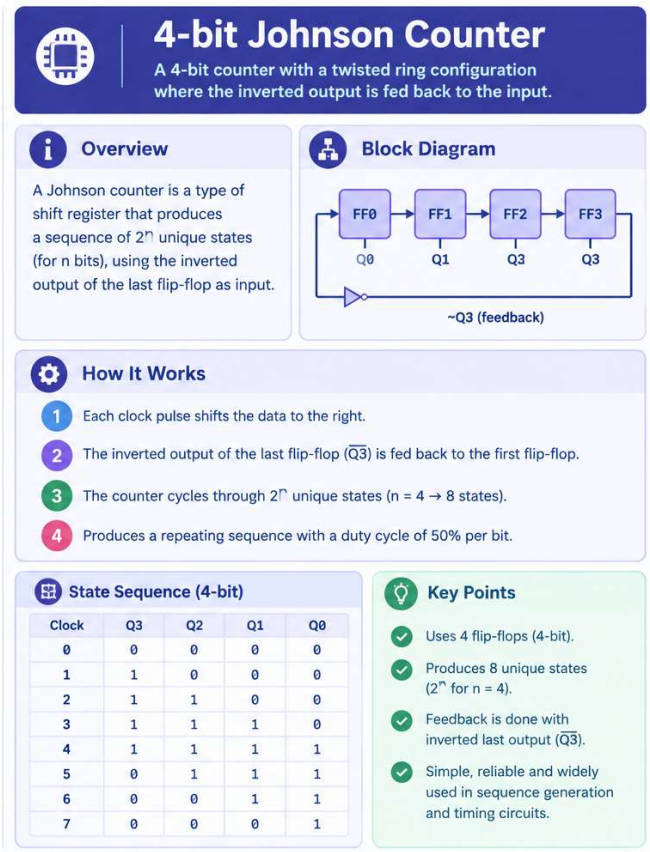
6. Johnson Counter

Theory

A Johnson counter (also known as a creeping counter, twisted ring counter, or Moebius counter) is a modified shift register counter where the inverted output of the final flip-flop is fed back into the serial input of the first flip-flop.Twisted Feedback: Unlike a standard ring counter that feeds back a non-inverted high signal, a Johnson counter takes the complement (inverse) of the last stage's output and loops it back.

- State Sequence: For an N-bit shift register, a Johnson counter generates $2N$ distinct states—double the number of states produced by a standard ring counter of the same length. For example, a 4-bit Johnson counter cycles through 8 unique states (e.g., transitioning from 0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 and back to 0000).
- Decoding Simplicity: Like ring counters, Johnson counters require very simple combinational decoding logic to generate specific timing pulses, making them highly efficient for multi-phase clock generators, stepper motor controllers, and waveform synthesis.

Design



Verilog HDL Code

```verilog
module tb_johnson_counter;

  reg       clk;
  reg       rst;
  reg [3:0] q;

  johnson_counter uut (
    .clk(clk),
    .rst(rst),
    .q(q)
  );

  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_johnson_counter);

    clk = 1'b0;
    rst = 1'b1;
    #12;

    rst = 1'b0;
    #100;

    $finish;
  end

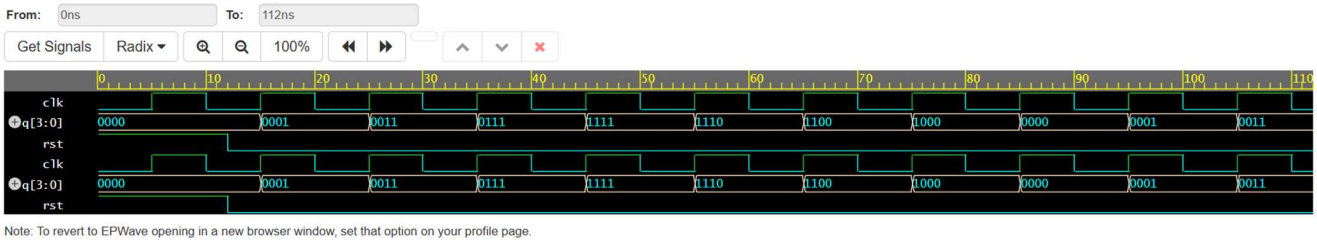
  initial begin
    $monitor("Time=%0t | rst=%b | q=%b", $time,
rst, q);
  end

endmodule
```

```verilog
module johnson_counter (
    input clk,
    inpu rst,
    output reg [3:0] q
);

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      q <= 4'b0000;
    end else begin
      q <= {q[2:0], ~q[3]};
    end
  end

endmodule
```



7. Synchronous and Asynchronous Counters

Theory

Digital counters are classified into two primary architectures based on how their internal flip-flops are clocked: Synchronous and Asynchronous (Ripple) counters.

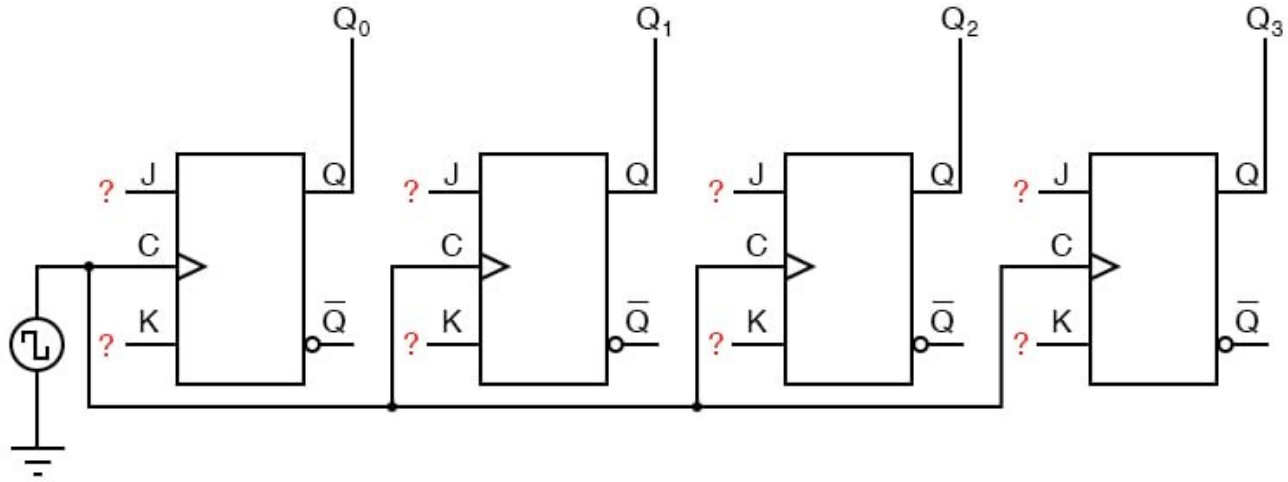
1. Synchronous Counters

    Every flip-flop in a synchronous counter is triggered simultaneously by a shared, common clock signal (clk). State transitions happen concurrently across all bits because the clock edge reaches all storage elements at the exact same time. Combinational logic gates (such as AND/OR gates) calculate the next state for each stage in parallel ahead of the clock edge.

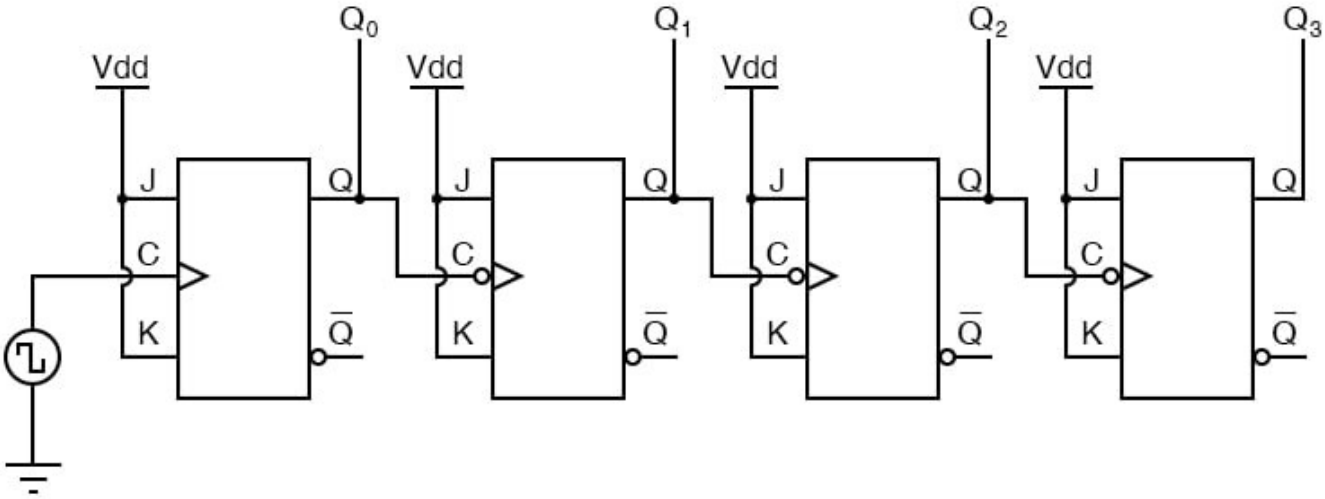
2. Asynchronous (Ripple) Counters

    Only the first flip-flop is driven by the external clock. Every subsequent flip-flop is clocked by the output transition (usually the $\bar{Q}$ or falling edge) of the preceding flip-flop. Changes "ripple" through the counter chain sequentially. When the first stage toggles, it triggers the next stage after a slight propagation delay, which in turn triggers the next.

Design



SYNCHRONOUS COUNTER



ASYNCHRONOUS COUNTER

Verilog HDL Code

```
module tb_sync_async_counters;

  reg       clk;
  reg       rst;
  reg [3:0] q_sync;
  reg [3:0] q_async;

  sync_async_counters uut (
    .clk(clk),
    .rst(rst),
    .q_sync(q_sync),
    .q_async(q_async)
  );

  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_sync_async_counters);

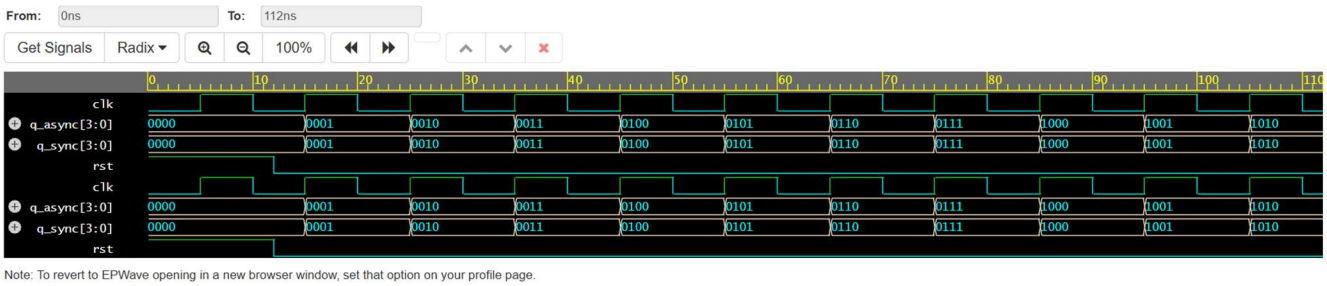
    clk = 1'b0;
    rst = 1'b1;
    #12;

    rst = 1'b0;
    #100;

    $finish;
  end

  initial begin
    $monitor("Time=%0t | rst=%b | q_sync=%b,
q_async=%b",
             $time, rst, q_sync, q_async);
  end
endmodule
```

```
1  module sync_async_counters (
2      input clk,
3      input rst,
4      output reg [3:0] q_sync,
5      output reg [3:0] q_async
6  );
7
8    always @(posedge clk or posedge rst) begin
9      if (rst) begin
10       q_async <= 4'b0000;
11     end else begin
12       q_async <= q_async + 1'b1;
13     end
14   end
15
16   always @(posedge clk) begin
17     if (rst) begin
18       q_sync <= 4'b0000;
19     end else begin
20       q_sync <= q_sync + 1'b1;
21     end
22   end
23
24 endmodule
```



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

8. Shift Register

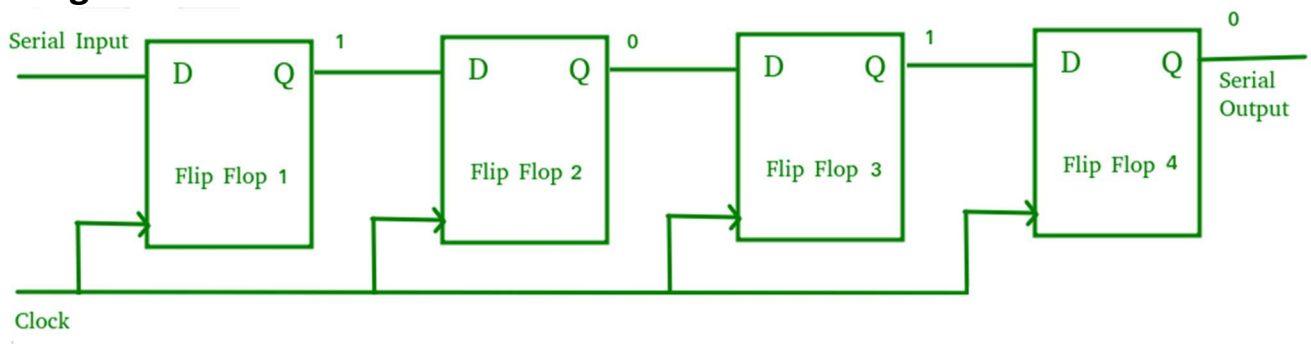
Theory

A shift register is a sequential digital logic circuit built by connecting multiple flip-flops (typically D-type) in a serial chain, sharing a common clock signal. Its primary function is to store, move, and manipulate digital data either

serially (one bit at a time per clock cycle) or in parallel.

- Serial-In, Serial-Out (SISO): Data enters one bit at a time and is shifted out sequentially from the final stage. Used primarily for temporary data storage or time delays.
- Serial-In, Parallel-Out (SIPO): Data is shifted in serially, but once a full word is loaded, all bits are read out simultaneously through parallel output lines. Commonly used for serial-to-parallel data conversion (e.g., UART receivers).
- Parallel-In, Serial-Out (PISO): A full data word is loaded into all flip-flops simultaneously via parallel control lines, and then shifted out one bit at a time. Used for parallel-to-serial conversion (e.g., transmitters).
- Parallel-In, Parallel-Out (PIPO): Data is loaded simultaneously and read out simultaneously on every clock cycle, serving as temporary buffer registers.

Design



Verilog HDL Code

```v
module tb_shift_register;

  reg       clk;
  reg       rst;
  reg       si;
  reg [3:0] po;

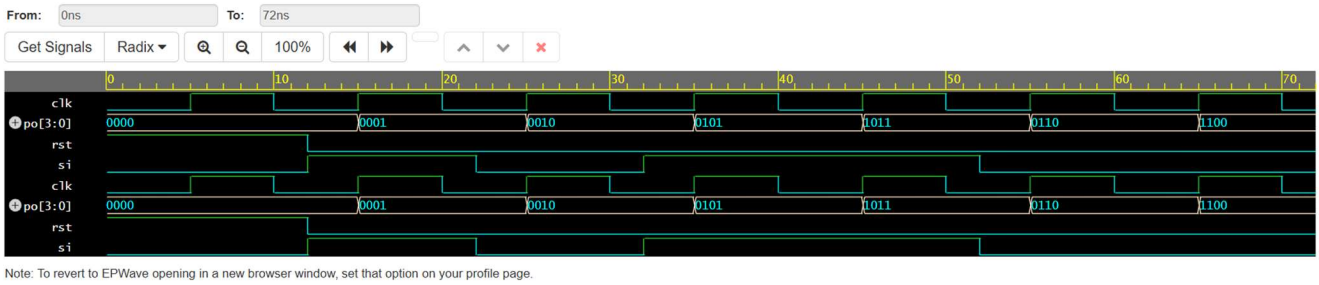
  shift_register uut (
    .clk(clk),
    .rst(rst),
    .si(si),
    .po(po)
  );
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_shift_register);
    clk = 1'b0;
    rst = 1'b1;
    si = 1'b0;
    #12
    rst = 1'b0;
    si = 1'b1; #10;
    si = 1'b0; #10;
    si = 1'b1; #10;
    si = 1'b1; #10;
    si = 1'b0; #20;

    $finish;
  end
  initial begin
    $monitor("Time=%0t | rst=%b, si=%b | po=%b",
$time, rst, si, po);
  end
endmodule
```

```v
module shift_register (
    input  reg       clk,
    input  reg       rst,
    input  reg       si,
    output reg [3:0] po
);

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      po <= 4'b0000;
    end else begin
      po <= {po[2:0], si};
    end
  end

endmodule
```



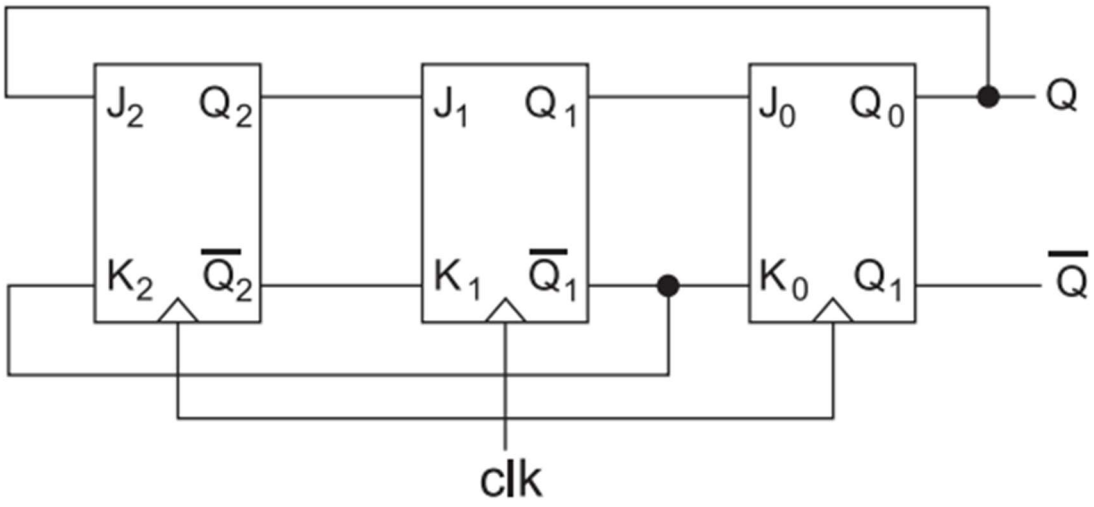
Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

9. Divide-by-N Counter

Theory

A divide-by-N counter (also known as a frequency divider) is a sequential digital logic circuit that takes an input clock signal and generates an output clock whose frequency is divided by an integer factor N. Built using a synchronous or asynchronous binary counter combined with a digital comparator or feedback decoding logic. The counter increments continuously on every active clock edge until it reaches a pre-configured

terminal count value ($N-1$). Upon reaching this limit, two primary actions occur:The counter resets its internal state back to zero on the subsequent clock cycle.An output toggle or a single-cycle pulse is generated, creating a new clock signal whose period is N times longer than the input clock period (thus scaling the frequency down by a factor of N).



Design

Verilog HDL Code

```
module tb_divide_by_n;

  reg       clk;
  reg       rst;
  reg       clk_out;

  divide_by_n #(.N(4)) uut (
    .clk(clk),
    .rst(rst),
    .clk_out(clk_out)
  );

  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_divide_by_n);

    clk = 1'b0;
    rst = 1'b1;
    #12;

    rst = 1'b0;
    #100;

    $finish;
  end

  initial begin
    $monitor("Time=%0t | rst=%b | clk_out=%b",
$time, rst, clk_out);
  end

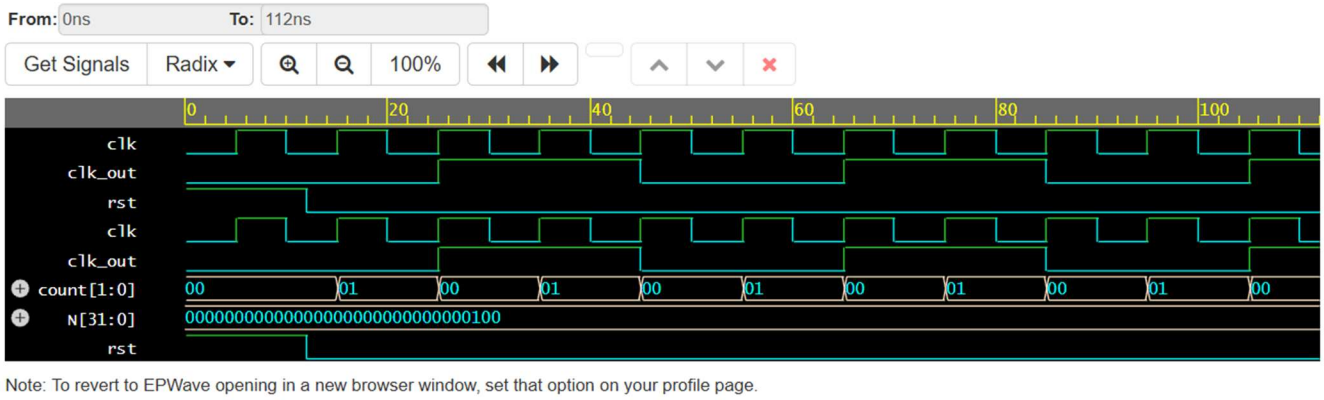
endmodule
```

```
module divide_by_n #(
    parameter N = 4
) (
    input clk,
    input rst,
    output reg clk_out
);

  reg [$clog2(N)-1:0] count;

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      count <= 0;
      clk_out <= 0;
    end else if (count == N/2 - 1) begin
      count <= 0;
      clk_out <= ~clk_out;
    end else begin
      count <= count + 1;
    end
  end

endmodule
```



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

10. Mealy FSM Example

Theory

A Mealy Finite State Machine (FSM) is a sequential digital logic system whose output depends on both its current internal state and the current

input values. This contrasts with a Moore machine, where outputs depend solely on the current state.

1. Core Operating Principles

- State Transition Logic: The next state is determined by combining the present state and current inputs using combinational logic.
- Output Generation: Outputs are updated immediately whenever an input changes or when the state transitions, often resulting in faster response times (fewer clock cycles of latency) compared to Moore machines.
- Asynchronous Outputs: Because Mealy outputs are combinatorial functions of both state and inputs, they can sometimes contain transient glitches if the input changes mid-clock cycle.

2. Sequence Detector (101 Overlapping Pattern)

A classic application of a Mealy FSM is detecting a specific bit stream, such as the sequence 101, with support for overlapping patterns (where the final 1 of one matched sequence serves as the starting 1 of the next sequence):

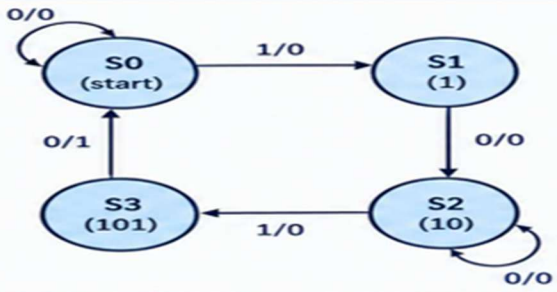
- State S0 (Idle / Reset): Waiting for the first 1. If input is 1, transition to S1; else remain in S0. Output is 0.
- State S1 (Saw '1'): Waiting for 0. If input is 0, transition to S2; if input is 1, stay in S1 (since a new sequence could start). Output is 0.
- State S2 (Saw '10'): Waiting for the final 1. If input is 1, the complete sequence 101 is detected, the output asserts high (1), and the machine transitions back to S1 to catch overlapping sequences. If input is 0, it resets back to S0.

Design

Mealy FSM (Finite State Machine)

- In a Mealy machine, the output depends on both the current state and the input (i.e., output = f(state, input)).
- Output is produced on the transitions (arrows).
- Usually has fewer states than a Moore machine for the same functionality.

Example: Mealy Sequence Detector (Detects 1010)

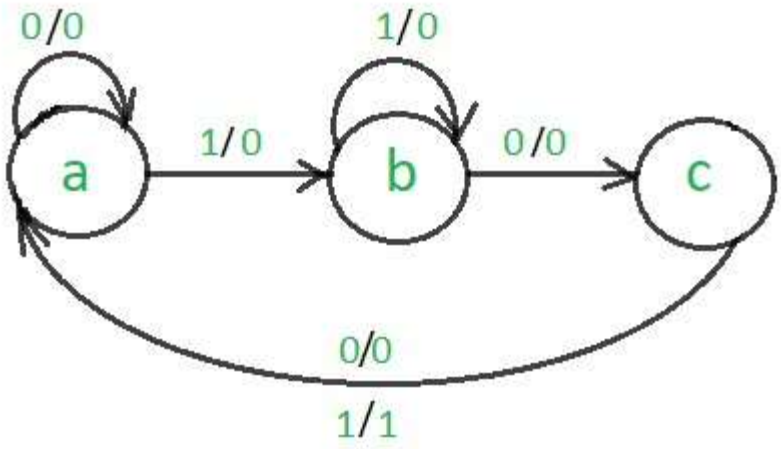


State Transition Table (Mealy)

Current State	Input = 0	Input = 1
	Next State / Output	Next State / Output
S0	S0 / 0	S1 / 0
S1	S2 / 0	S1 / 0
S2	S2 / 0	S3 / 0
S3	S0 / 1	S1 / 0

Explanation

- The machine detects the sequence 1010.
- When the sequence is detected (transition S3 → S0 with input 0), output = 1.
- Output is written on the transition labels (x/y).



Verilog HDL Code

```
module tb_moore_101_detector;
  reg       clk;
  reg       rst;
  reg       in;
  reg       out;
  moore_101_detector uut (
    .clk(clk),
    .rst(rst),
    .in(in),
    .out(out)
  );
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_moore_101_detector);
    clk = 1'b0;
    rst = 1'b1;
    in = 1'b0;
    #12
    rst = 1'b0;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    $finish;
  end
  initial begin
    $monitor("Time=%0t | rst=%b, in=%b | out=%b",
$time, rst, in, out);
  end
endmodule
```

```
module moore_101_detector (
    input  reg       clk,
    input  reg       rst,
    input  reg       in,
    output reg       out
);

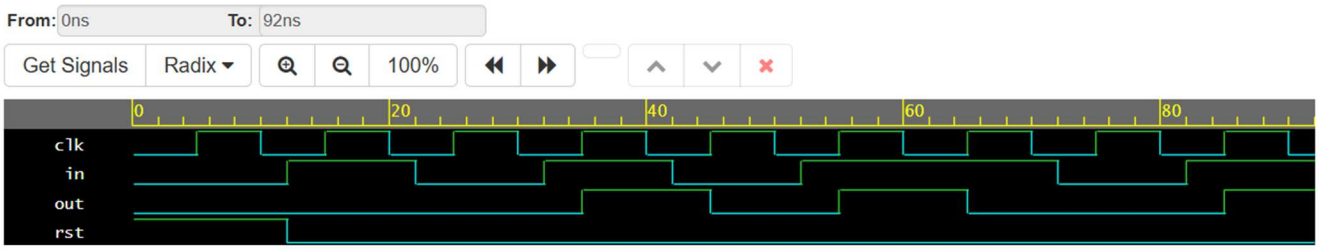
  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
  reg [1:0] state, next_state;

  always @(posedge clk or posedge rst) begin
    if (rst)
      state <= S0;
    else
      state <= next_state;
  end

  always @(*) begin
    case (state)
      S0: next_state = in ? S1 : S0;
      S1: next_state = in ? S1 : S2;
      S2: next_state = in ? S3 : S0;
      S3: next_state = in ? S1 : S2;
      default: next_state = S0;
    endcase
  end

  always @(*) begin
    out = (state == S3);
  end

endmodule
```



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

11. Moore FSM Example

Theory

A Moore Finite State Machine (FSM) is a sequential digital logic system whose output values depend solely on its current internal state, regardless of the current input values. This contrasts with a Mealy machine, where outputs are determined by both the current state and inputs.

1. Core Operating Principles

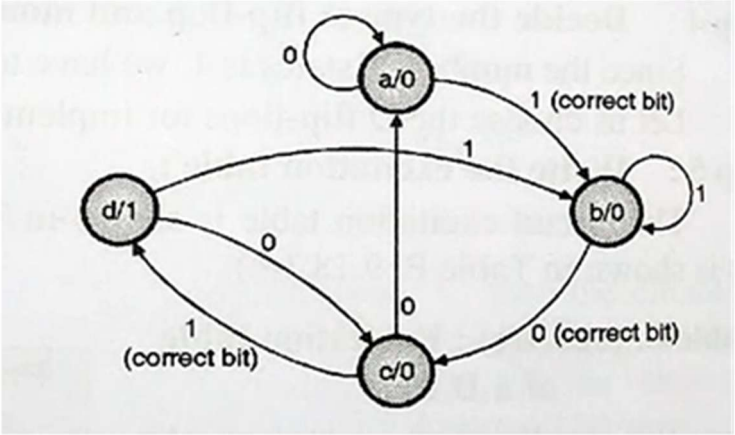
- State Transition Logic**:** The next state is governed by combinational logic evaluating the present state and input signals on each active clock edge.
- Synchronous Outputs: Because Moore outputs depend exclusively on the state register, they change only on clock edges. This makes the output signals immune to combinatorial input glitches that might occur mid-cycle.
- Clock Cycle Latency: Moore outputs typically experience a one-clock-cycle delay in reflecting an input change compared to Mealy machines, as the system must transition into the matching output state first.

2. Sequence Detector (101 Pattern)

A classic application of a Moore FSM is detecting a specific serial bit stream, such as the sequence 101:

- State S0 (Idle / Reset): Waiting for the initial 1. If input is 1, transition to S1; else remain in S0. Output is 0.
- State S1 (Saw '1'): Waiting for 0. If input is 0, transition to S2; if input is 1, stay in S1. Output is 0.
- State S2 (Saw '10'): Waiting for the final 1. If input is 1, transition to S3 (success state); if input is 0, reset back to S0. Output is 0.
- State S3 (Sequence 101 Detected): The complete sequence has been successfully identified, asserting the output high (dout = 1). From here, depending on whether overlapping sequences are supported, the FSM transitions back to evaluate subsequent bits.

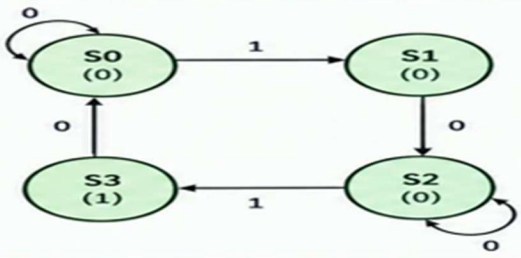
Design



Moore FSM (Finite State Machine)

- In a Moore machine, the output depends only on the current state (i.e., output = f(state)).
- Output is associated with the states (not the transitions).
- Often requires more states than a Mealy machine for the same functionality.

Example: Moore Sequence Detector (Detects 1010)



State Transition Table (Moore)

Current State (Output)	Input = 0	Input = 1
	Next State	Next State
S0 (0)	S0	S1
S1 (0)	S2	S1
S2 (0)	S2	S3
S3 (1)	S0	S1

Explanation

- The machine detects the sequence 1010.
- When the sequence is detected, the machine enters state S3, which has output = 1.
- Output is associated with the state (shown inside the state).

Verilog HDL Code

```v
module tb_moore_101_detector;
  reg       clk;
  reg       rst;
  reg       in;
  reg       out;
  moore_101_detector uut (
    .clk(clk),
    .rst(rst),
    .in(in),
    .out(out)
  );
  always #5 clk = ~clk;
  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_moore_101_detector);
    clk = 1'b0;
    rst = 1'b1;
    in = 1'b0;
    #12;
    rst = 1'b0;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    in = 1'b1; #10;
    in = 1'b0; #10;
    in = 1'b1; #10;
    $finish;
  end
  initial begin
    $monitor("Time=%0t | rst=%b, in=%b | out=%b",
$time, rst, in, out);
  end
endmodule
```

```v
module moore_101_detector (
    input  reg       clk,
    input  reg       rst,
    input  reg       in,
    output reg       out
);

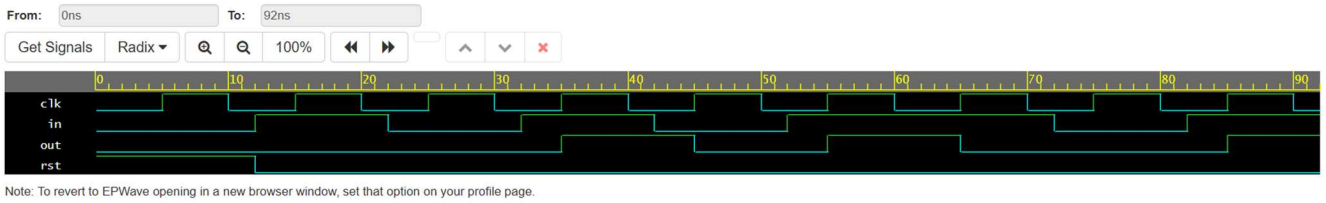
  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
  reg [1:0] state, next_state;

  always @(posedge clk or posedge rst) begin
    if (rst)
      state <= S0;
    else
      state <= next_state;
  end

  always @(*) begin
    case (state)
      S0: next_state = in ? S1 : S0;
      S1: next_state = in ? S1 : S2;
      S2: next_state = in ? S3 : S0;
      S3: next_state = in ? S1 : S2;
      default: next_state = S0;
    endcase
  end

  always @(*) begin
    out = (state == S3);
  end

endmodule
```



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

12. Random Sequence Generator

Theory

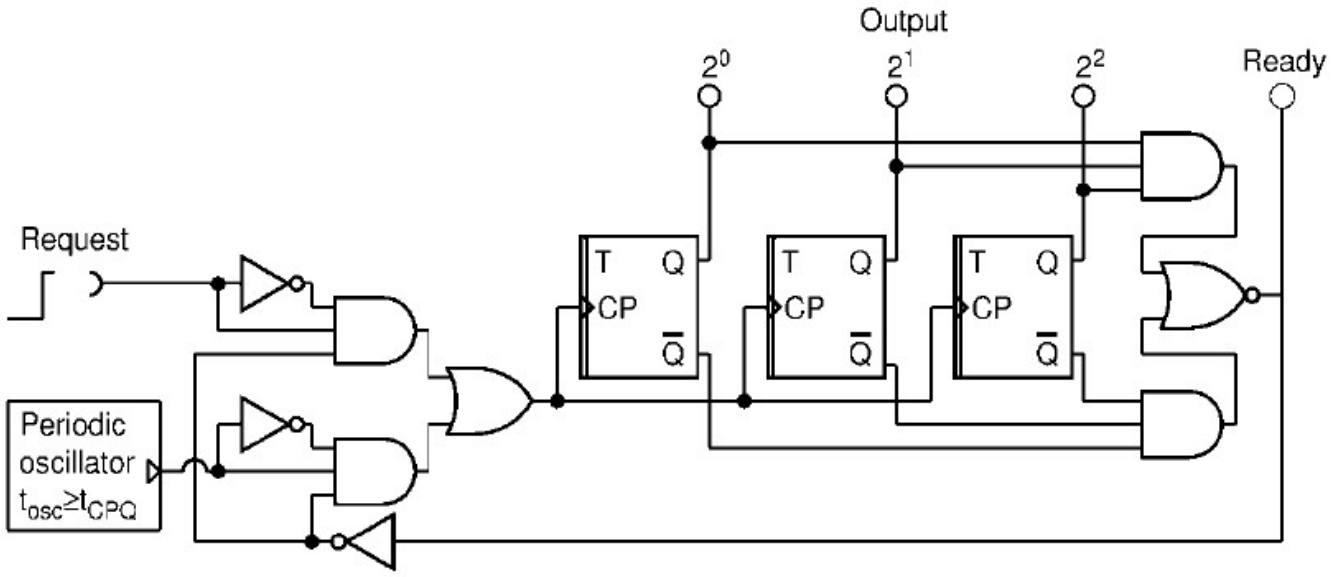
A random sequence generator in digital hardware typically refers to a Pseudo-Random Binary Sequence (PRBS) generator, most commonly implemented using a Linear Feedback Shift Register (LFSR).

Operational Mechanics

- Shift Register Chain: The circuit consists of a chain of flip-flops connected serially, sharing a common clock signal.

- Feedback Network: The input to the first flip-flop is derived by tapping specific intermediate bit positions and combining them using Exclusive-OR (XOR) or XNOR logic gates. The specific combination of tap points defines the generator's characteristic polynomial.
- Pseudo-Random Output: On each clock edge, the bits shift down the register while the feedback network computes the next input bit. For an N-bit register with carefully chosen taps (maximal-length configuration), the circuit cycles through $2^N - 1$ unique pseudo-random states before repeating the sequence.

Design



Verilog HDL Code

```verilog
module tb_random_seq_generator;

  reg       clk;
  reg       rst;
  reg [3:0] lfsr_out;

  random_seq_generator uut (
    .clk(clk),
    .rst(rst),
    .lfsr_out(lfsr_out)
  );

  always #5 clk = ~clk;

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1, tb_random_seq_generator);

    clk = 1'b0;
    rst = 1'b1;
    #12;

    rst = 1'b0;
    #160;

    $finish;
  end

  initial begin
    $monitor("Time=%0t | rst=%b | lfsr_out=%b",
$time, rst, lfsr_out);
  end

endmodule
```

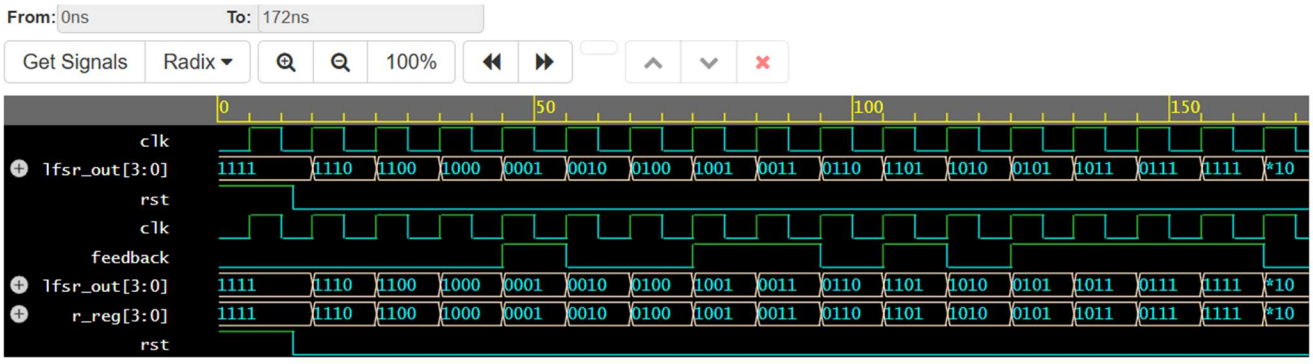
```verilog
module random_seq_generator (
    input  reg       clk,
    input  reg       rst,
    output reg [3:0] lfsr_out
);

  reg [3:0] r_reg;
  reg       feedback;

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      r_reg <= 4'b1111;
    end else begin
      feedback = r_reg[3] ^ r_reg[2];
      r_reg <= {r_reg[2:0], feedback};
    end
  end

  always @(*) begin
    lfsr_out = r_reg;
  end

endmodule
```



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.